

モダンSaaS開発 & プロジェクト推進 総合プレイブック

GitHub × Vercel × Supabase を用いた高速開発・ゼロ待ち時間UX・採算管理・やること管理ノウハウ

分類: 汎用Webサービス・生成AI SaaS開発標準ガイド | 最終更新: 2026年10月1日（第2版） |

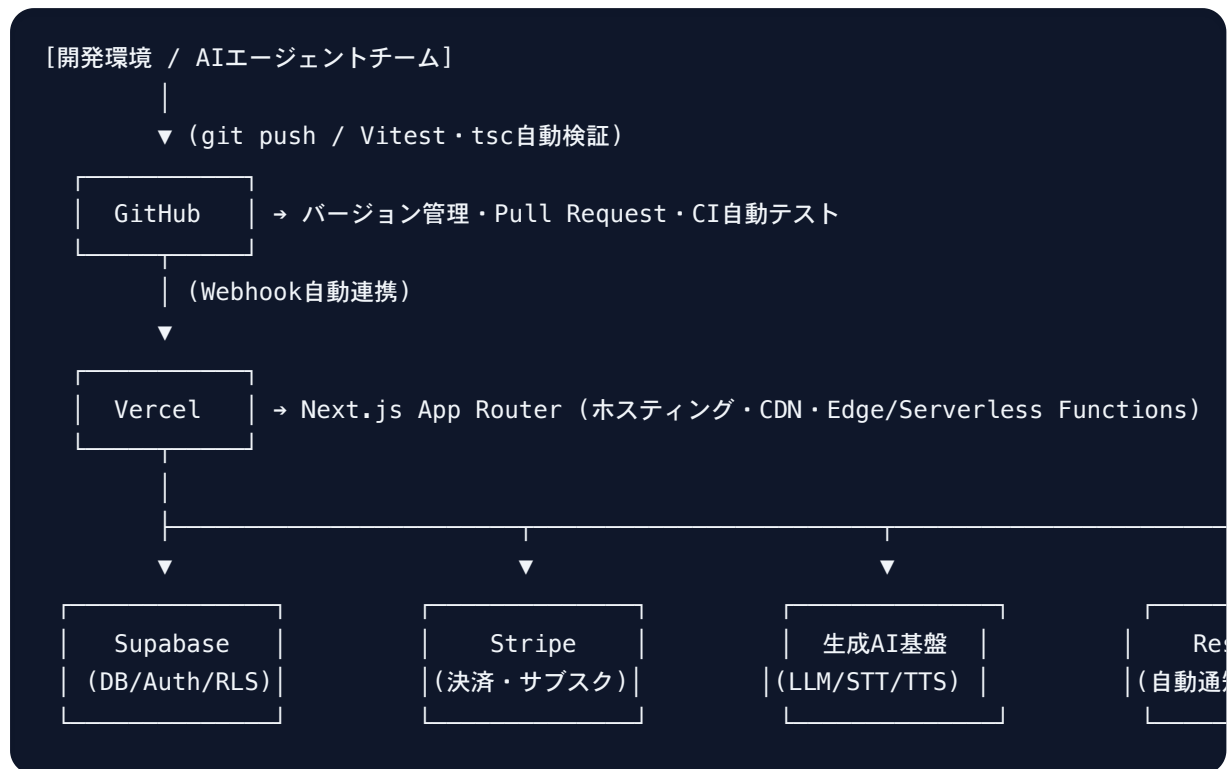
推奨環境: Next.js 15 / TypeScript / Supabase / Stripe / Resend

【本ドキュメントの目的】

本ドキュメントは、Next.js、GitHub、Vercel、Supabase、および外部決済・生成AI APIを組み合わせてWebサービス（SaaS）を迅速かつ安全に立ち上げ・スケールさせるための**システム設計アーキテクチャ（0.0秒先読み・コンテンツバンク・自動レスキュー同期）**および**プロジェクト推進（4ロール分業・勝手に直さないガバナンス・抜け漏れ防止4大プロトコル・2フェーズ原価管理）**を体系化した汎用プレイブックです。特定のサービス名に依存せず、新規事業の立ち上げ時に標準テンプレートとしてそのまま活用できます。

1 システム構造 & ゼロ待ち時間アーキテクチャ

1. 全体パイプライン (GitHub → Vercel → Supabase)



2. オフラインファースト & 未同期データの「自動レスキュー同期」設計

- ① **ゲスト体験 (登録不要):** 最初の数回は登録不要で即時利用可能にし、LocalStorageへ一時保存して離脱を防止。
- ② **リアルタイム保存 + 自動レスキュー同期:** アクション完了時にローカル保存と同時にSupabaseへ即時INSERT。さらに画面起動時に「ローカルにあるがクラウドに未反映のレコード (localOnly)」を自動検出し、バックグラウンドでクラウドへ救出同期する。

3. 生成AIの「5秒待ち」を「0.0秒」にするハイブリッド先読み&バンク設計

LLMや音声合成 (TTS) はリアルタイム生成すると3~5秒の待ち時間が発生し、API原価もかさみます。以下の2段階設計で「体感待ち時間 0.0秒」と「API変動原価 70~80%削減」を同時達成します。

⚡ ① クライアント1件先読み (Prefetch)

1問目を操作中に裏側 (非同期) で「次の1問 (テキスト+音声)」を先読みしてメモリ保持。「次へ」を押した瞬間に待ち時間 0.0秒 で即時表示。

🏦 ② DBバンク蓄積 & 週次10%入れ替え

ユーザー利用の裏で生成データをDB (各カテゴリ上限300件) に蓄積。上限到達後はDBから0.1秒・原価0円で配信し、毎週1回「使用回数上位10%」だけを季節・最新トレンド入りで入れ替え。

2 セキュリティ & データ整合性の鉄則

1. Row Level Security (RLS) & 管理者権限の単一厳格化

全テーブルでRLSを有効化し、本人のみ読み書きを許可。管理者画面（/admin/*）やテストモードの判定（isAdminEmail）は、開発用ダミーアドレスを残さず**本番オーナーの単一メールアドレスのみに厳格化**する。

```
ALTER TABLE public.user_data ENABLE ROW LEVEL SECURITY;  
CREATE POLICY "User owns data" ON public.user_data FOR ALL USING (auth.uid() =  
user_id);
```

2. 決済情報の完全非保持化 (PCI-DSS準拠)

- **Stripe Checkout / Customer Portal** の全面採用により、カード情報はStripe内で完結。
- Webhookイベント（課金成功・解約・期限切れ）を受信してSupabaseのプランステータスを自動更新。
- 特定商取引法に基づく表記（/tokushoho）は全画面フッターで目立たせず、決済モдал・ログイン導線に適切配置。

3. サーバーレス環境での永続化と環境変数多段フォールバック

VercelのServerless Functionsは数分でコンテナが破棄されるため、アクセス解析（PV/UU）等はメモリ変数（Map）ではなく必ずSupabase（daily_analytics等）へ日次UPSERT永続化する。また、Vercel連携時の環境変数重複プレフィックスに備え、初期化コードには多段キーフォールバックを実装する。

1. 4ロール分業体制（オーナー × 統括PM × 技術相談 × 実務実装）

① オーナー / PO

最高意思決定
事業方針・最終承認

② 統括担当 / PM

進捗/FB管理・原価試算
週次監査・仕様書統括

③ 技術相談・設計

技術壁打ち・UX検証
詳細実装指示書(SPEC)

④ 実務・実装担当

コード実装・Vitest検証
ビルド確認・デプロイ

【鉄則1】「勝手に直さない」意思決定プロトコル

1. **検知**: 問題や不整合箇所を特定。
2. **提案**: 修正案A/Bと影響を整理して提示。
3. **判断**: オーナーの承認を得る。
4. **着手**: 承認後に実装・テスト・報告。

【鉄則2】抜け漏れゼロの「4大プロトコル」

1. **即時復唱**: 会話の場で優先度と時期を合意。
2. **即時起票**: その場で FEEDBACK_LOG.md へ記録。
3. **定期スキャン**: 毎週月曜朝に未着手項目を点検。
4. **HTML可視化**: 統括ダッシュボードへ即時同期。

🟢 本番環境を汚さない「管理者限定プレビュー（Feature Toggle）」方式:

新UIや実験的機能（例: チャンク区切り表示や新録音方式）は、まず管理者アカウント（isAdminEmail / 🟢 テストボタンモード）限定で本番に表示してオーナーが実機検証し、完成度を確認してから全ユーザー標準ONへ昇格させる。

2. ドキュメント体系の標準テンプレート（5点セット）

ファイル名	役割・記録内容	更新頻度
PROGRESS_MEMO.md	全体進捗、決定事項ログ、直近ToDoの一元管理	決定事項発生ごと
FEEDBACK_LOG.md	顧客・テスターの声（VoC）と対応ステータス管理簿	FB受領・相談ごと
weeklylog.md	週次での決定事項・実装・直面したつまづきと解決策のクロニクル記録	毎週月曜朝＋節目
SYSTEM_SPECIFICATION.md	サービスの基本仕様・価格・DB・2フェーズ原価・ルールのマスター設計書	仕様変更時（即時）
XXXX_SPEC.md	機能単位の詳細実装指示書（背景、要件、管理者フラグ、テスト方針）	機能着手ごと

1. 2フェーズ原価管理モデル（蓄積期 → バンク定常運用期）

生成AIを組み込むSaaSでは、「蓄積期（Phase A）」と「バンク定常期（Phase B）」の2段階で原価と営業利益率を管理します。

📖 フェーズA：バンク蓄積期（保守上限）

各スロットが上限（300件）に達するまでの期間。都度生成＋解析で1回あたり約**0.322円**。
50名規模での営業利益率は約**55～58%**。管理画面の安全監視基準に採用。

🏠 フェーズB：バンク定常期（本命）

上限到達後はDBから0円で配信し、解析のみ（約**0.097円/回**、原価70%超削減）＋週10%入替（月額固定約**331円**）。営業利益率は約**79～80%**へ向上！

2. 顧客管理画面（/admin/customers）の4大モニタリング機能

有料プランを「使い放題」として提供しつつ、管理画面で各顧客の利用ペース・日次推移・監査ログを一元監視します。

月間損益分岐回数 = (月額料金 - Stripe手数料 3.6%) ÷ 1回原価 / 1日限界回数 = 月間損益分岐回数 ÷ 30日

● 安全 (Safe)

限界の50%未満利用

● 注意 (Warning)

限界の50%～80%利用

● 警戒 (Danger)

限界の80%以上利用

- メンバー別「日次利用回数折れ線グラフ」モダール: 利用回数ボタンから日別推移グラフ＋合計・平均・最高記録を即時表示。
- プラン別内訳（Pro/Basic） & モバイルカード表示切替 & コンプライアンス監査ログタブ統合。

Stage 0: MVP構築 & コア検証

- ☑ Next.js + Tailwind フロントエンド構築
- ☑ コアドメイン機能（提供価値）の動的検証
- ☑ ゲスト体験（LocalStorage一時保存）
- ☑ 単体・統合テスト環境（Vitest / tsc）

Stage 1: 課金・認証・法務

- ☑ Supabase Auth (Email / Google) 導入
- ☑ DBスキーマ設計 & RLSポリシー完全適用
- ☑ Stripe Checkout / Webhook 決済基盤
- ☑ 法務3画面 (/terms, /privacy, /tokushoho)

Stage 2: クローズドβ・LP集客 & 0.0秒化

- ☑ 動くデモ付き先行LP (/waitlist) & クーポン自動返信
- ☑ β期間限定VIP（無制限）自動付与キャンペーン
- ☑ 1問先読み (0.0秒遷移) & DBバンク週次10%入替
- ☑ 管理者プレビューモード & 問い合わせ自動通知

Stage 3: ローンチ & 収益化・自走運用

- ☑ SNS（Threads等）発信 & インサイト連携
- ☑ 顧客管理（採算リスク・日次折れ線グラフ・監査）
- ☑ 未同期データの自動レスキュー同期パイプライン
- ☑ 毎週月曜朝の定期点検（weeklylog更新 & 監査）